SECOS Backend Documentation (FastAPI)

AI-Powered Smart Energy Optimization System: Backend Service Design, v1.0

1. Purpose and Scope

SECOS is a mobile application that helps Egyptian households reduce electricity waste. It forecasts future consumption, warns users before they cross into a higher tariff tier, and recognizes appliances and electricity-app screenshots through computer vision.

This document describes what the **backend** will do. The backend is the orchestrator: it is the only component the mobile app talks to, and the only component that talks to the ML Engine and the CV Module. It owns users, data storage, Egyptian tariff-tier logic, scheduling and alerts.

In scope: REST API, authentication, database, orchestration of ML and CV, tier calculation, daily forecasting job, push alerts, recommendations.

Out of scope: model training, UI, and IoT or real-time metering (listed as future work).

2. Architecture Overview

The system is split into four decoupled domains. They communicate only through JSON over HTTP, and the backend orchestrates every exchange.

Domain	Responsibility	Input	Output
Mobile App (Frontend)	UI, camera capture	User taps, camera frames	JSON requests to Backend
Backend (FastAPI)	Orchestration, tiering, storage, alerts	App requests, ML forecasts, CV results	Aggregated JSON, push notifications
ML Engine	Time-series forecasting (stateless)	Weather, profile, lag values	7 to 14 day predicted kWh array
CV Module	OCR and appliance recognition	Raw images (.jpg, .png)	Text labels or numeric kWh

Key rules

- The mobile app never calls the ML Engine or CV Module directly.
- The ML Engine never calculates currency or tiers. It only maps weather and profile data to predicted kWh.
- All tariff and tier logic lives in the backend.

3. Technology Stack

Layer	Choice	Notes
Framework	FastAPI (Python 3.11+)	Async, automatic OpenAPI docs
Validation	Pydantic v2	Request and response schemas
Database	PostgreSQL	SQLite is acceptable for early development
ORM and migrations	SQLAlchemy 2.x and Alembic	
Auth	JWT (access and refresh tokens), bcrypt password hashing	
Scheduler	APScheduler or Celery Beat	Daily forecasting cron job
HTTP client	httpx (async)	Calls to ML Engine, CV Module and weather API
Push alerts	Firebase Cloud Messaging (FCM)	
Weather source	Any provider with a daily forecast API (for example OpenWeatherMap)	
Deployment	Docker and docker-compose	Backend, ML and CV as separate containers

4. Suggested Project Structure

```
secos-backend/
  app/
    main.py              # FastAPI app, router registration
    core/                # config, security, logging
    api/v1/              # auth.py, users.py, devices.py, calibration.py,
```

```
                        # forecast.py, alerts.py, recommendations.py
    schemas/            # Pydantic models (incl. ML contract)
    models/             # SQLAlchemy models
    services/
      ml_client.py      # calls ML Engine
      cv_client.py      # calls CV Module
      weather_client.py  # weather provider
      tier_service.py   # Egyptian tariff logic
      forecast_service.py
      alert_service.py
    jobs/               # daily_forecast.py
    db/                 # session, base, migrations
  tests/
  Dockerfile
  docker-compose.yml
```

5. System Data Flow

Phase 1: Cold Start and Calibration

1. The app collects location (see open point 1), home area in square meters, household size and a screenshot of the user's electricity app.
2. The backend sends the screenshot to the CV Module, which extracts the historical kWh baseline.
3. The backend computes a **calibrated multiplier** for the user's profile and stores it.

Phase 2: Daily Forecasting and Tier Alerts

1. A cron job runs once per day for every active user.
2. The backend fetches the weather forecast and calls the ML Engine with the user's profile, history and weather.
3. The ML Engine returns 7 to 14 days of predicted kWh.
4. The backend adds the predictions to the user's current-month usage, compares the projected total against the Egyptian tier thresholds, and detects an imminent tier jump.
5. If a jump is projected, the backend stores an alert and sends a push notification to the app.

Phase 3: Appliance Recognition (on demand)

1. The user photographs an appliance.
2. The backend forwards the image to the CV Module and receives the device type.
3. The backend maps the device type to its specification (typical wattage), stores it, and returns usage tips.

6. Backend API (Proposed Endpoints)

All routes are prefixed with `/api/v1` and, except for auth, require a Bearer JWT.

Method	Endpoint	Description
POST	/auth/register	Create an account
POST	/auth/login	Returns access and refresh tokens
POST	/auth/refresh	Refreshes the access token
GET / PUT	/users/me	Read or update the profile
PUT	/users/me/household	Save area, household size and location
POST	/calibration/screenshot	Upload an app screenshot (CV OCR, then baseline and multiplier)
POST	/consumption	Add a manual usage reading
GET	/consumption?from=&to=	Usage history
GET	/forecast	Latest 7 to 14 day forecast with projected tier
POST	/forecast/refresh	Force a forecast now (rate limited)
GET	/tiers/status	Current tier, kWh to next tier, estimated cost
POST	/devices/recognize	Upload a photo, receive the appliance type
GET / POST / DELETE	/devices	Manage the user's devices

Method	Endpoint	Description
GET	/recommendations	Personalized saving tips
GET	/alerts	Alert list; PATCH /alerts/{id} marks as read
POST	/devices/push-token	Register an FCM token
GET	/health	Liveness check

7. ML Engine Contract (Backend as Client)

The ML Engine is a stateless microservice. Endpoint: `POST /api/v1/ml/predict`.

Request (Backend to ML Engine)

```
{
  "user_id": "usr_98231",
  "timestamp": "2026-09-18T15:30:00Z",
  "forecast_days": 7,
  "household_metadata": {
    "square_footage": 120,
    "household_size": 4,
    "calibrated_multiplier": 1.15
  },
  "historical_consumption": {
    "previous_month_total_kwh": 420.0,
    "last_recorded_daily_kwh": 14.2
  },
  "weather_forecast": [
    {"date": "2026-09-19", "temp_max": 38, "humidity": 55, "condition":
"sunny"}
  ]
}
```

Response (ML Engine to Backend)

```
{
  "status": "success",
  "model_version": "v1.2-linear-regression",
  "predictions": [
    {"date": "2026-09-19", "predicted_kwh": 14.8}
  ]
}
```

Backend responsibilities for this integration

- Define Pydantic models that mirror this contract exactly, and validate the ML response before using it.
- Use a timeout (suggested: 5 seconds) with one retry. If the ML Engine fails, fall back to the last successful forecast and flag it as stale.
- Store `model_version` with each forecast for traceability.
- Make the Linear Regression to Random Forest upgrade invisible to the app by keeping the contract unchanged.

8. CV Module Contract (Backend as Client)

The contract is not yet fixed in the specification, so the following is a proposal to agree with the CV teammate.

- `POST /api/v1/cv/ocr` accepts an image and returns `{ "status", "extracted_kwh", "period", "confidence" }`.
- `POST /api/v1/cv/classify` accepts an image and returns `{ "status", "device_type", "confidence" }`.
- Accept only `.jpg` and `.png`, with a maximum size of about 5 MB.
- If confidence is below a threshold (for example 0.6), return a "please confirm or enter manually" response to the app instead of saving the result.

9. Egyptian Tier Logic (`tier_service`)

The backend converts kWh into tiers and cost. The ML Engine never does this.

- Store the tariff table in **configuration or database, not in code**: tier ranges in kWh per month, price per kWh, and fixed fees. Tariffs change, so confirm the current official values before implementation.

- Algorithm: `projected_month_kwh = kwh_used_so_far + sum(predicted_kwh for remaining days)`. Find the tier of the current usage and of the projection. If the projected tier is higher, create an alert that includes the date the threshold will likely be crossed and how many kWh remain below it.
- Because the specification names the region "Prepaid/Tiered", also support the prepaid case: remaining balance divided by predicted daily cost gives an estimated days-remaining figure.
- Keep this module pure (no I/O) so it is easy to unit test.

10. Database Design

Table	Key columns
users	id, email, password_hash, created_at
households	id, user_id, location, area_m2, household_size, calibrated_multiplier
consumption_records	id, user_id, date, kwh, source (manual, ocr, estimated)
forecasts	id, user_id, generated_at, model_version, is_stale
forecast_days	id, forecast_id, date, predicted_kwh
devices	id, user_id, name, type, wattage, image_url
alerts	id, user_id, type, message, created_at, read
tariff_tiers	id, min_kwh, max_kwh, price_per_kwh, effective_from
push_tokens	id, user_id, token, platform

Index `consumption_records (user_id, date)` and `forecast_days (forecast_id, date)`.

11. Background Jobs

Job	Schedule	Action
daily_forecast	Once per day (early morning)	Fetch weather, call ML, store forecast, evaluate tiers, send alerts
stale_forecast_cleanup	Weekly	Remove old forecasts
token_cleanup	Daily	Remove expired refresh tokens

Process users in batches with limited concurrency so a slow weather provider or ML call does not block everyone.

12. Security and Privacy

- Passwords hashed with bcrypt, JWT with short-lived access tokens, HTTPS only.
- Secrets in environment variables, never in the repository.
- Every query is scoped by the authenticated `user_id`, so one user can never read another's data.
- Validate uploaded images (type, size) and strip metadata before storing.
- Rate limit login, upload and forecast-refresh endpoints.
- Provide data deletion on request, and keep recommendations explainable (state why a tip is suggested), as the project's ethical considerations require.

13. Error Handling and Conventions

Use a consistent error body: `{ "error": { "code": "...", "message": "..." } }`.

Case	Status
Validation failure	422
Unauthorized	401
Upstream (ML, CV, weather) failed	502, or a cached fallback where possible
Upstream timeout	504
Low-confidence OCR	200 with `needs_confirmation: true`

Log each request with a correlation ID that is also passed to the ML and CV calls.

14. Targets (From the Project KPIs)

Metric	Target	Backend implication
Prediction accuracy	above 85%	Log predictions against actuals for evaluation
Response time	under 2 seconds	Serve `/forecast` from stored results; never call ML inside a user request unless refreshing
Energy reduction	20 to 30%	Track alerts acted upon and month-over-month usage
User engagement	High retention	Useful, not noisy, alerts

15. Testing Strategy

- **Unit tests:** tier_service, multiplier calculation, schema validation.
- **Integration tests:** API routes with a test database, with the ML and CV clients mocked.
- **Contract tests:** validate ML and CV responses against the agreed schemas.
- **Load test:** confirm the under-2-second response target for the main read endpoints.
- Tools: pytest, pytest-asyncio, httpx TestClient.

16. Suggested Backend Timeline

Weeks	Backend work
1 to 3	Requirements, database schema, API design, FastAPI skeleton
4 to 10	Auth, profiles, consumption, tier service, ML client with mocks, daily job
10 to 13	Integrate real ML and CV services, push alerts, recommendations
14 to 16	Testing, performance tuning, deployment, documentation

17. Open Points to Resolve With the Team

1. **Location input:** the specification asks for a ZIP code, which is not commonly used in Egypt. Consider governorate or city, which the weather API can also use.
2. **Square footage versus meters:** the sample payload uses `square_footage: 120`, which looks like square meters. Agree on one unit.
3. **Platform mismatch:** the idea document says "web dashboard", while the project is a mobile app. The architecture here assumes the mobile app is the frontend.
4. **CV contract:** finalize the request and response formats in section 8 with the CV teammate.
5. **Model name:** the ML sample returns `v1.2-linear-regression`. Confirm when Random Forest replaces it.
6. **Tariff values:** confirm the official tier table and whether prepaid meters are in scope.